

GESTIÓN AUTOMÁTICA FRENTE ATAQUES

PRY

Gestión de Redes

GESTIÓN AUTOMÁTICA FRENTE ATAQUES

Índice

1 Introducción	2
2 Descripción del sistema	2
3 Necesidades de gestión	3
4 Planificación	4
5 Solución de gestión	5
5.1 ÁREA DE GESTIÓN DE ANOMALÍAS:	5
5.2 ÁREA DE GESTIÓN DE CONTROL(MONITORIZACIÓN)	10
5.3 CONEXIÓN ENTRE GESTIÓN DE ANOMALÍAS Y GESTIÓN DE CONTROL:	15
5.4 ÁREA DE AUTOMATIZACIÓN	19
5.5 AUTOESCALADO	23
5.6 PUESTA EN MARCHA DEL SISTEMA	27
5.7 CONCLUSIONES	32
6 Bibliografía	33



Universidad
de Alcalá

1 INTRODUCCIÓN

En el contexto actual, los servicios online sufren constantes intentos de ataque que pueden saturar los servidores y detener el negocio. La defensa manual mediante operadores físicos es lenta ante ataques por la movilización de equipos enteros.

Por eso decidimos que nuestro objetivo debería ser un proyecto para automatizar la seguridad mediante un sistema de alertas basado en Inteligencia Artificial. Buscamos que el sistema sea capaz de 'leer' el tráfico de red y, al detectar anomalías, realizar un autoescalado inmediato. Esto permite liberar la red y mitigar el ataque de forma autónoma, eliminando la necesidad de supervisión humana constante.

2 DESCRIPCIÓN DEL SISTEMA

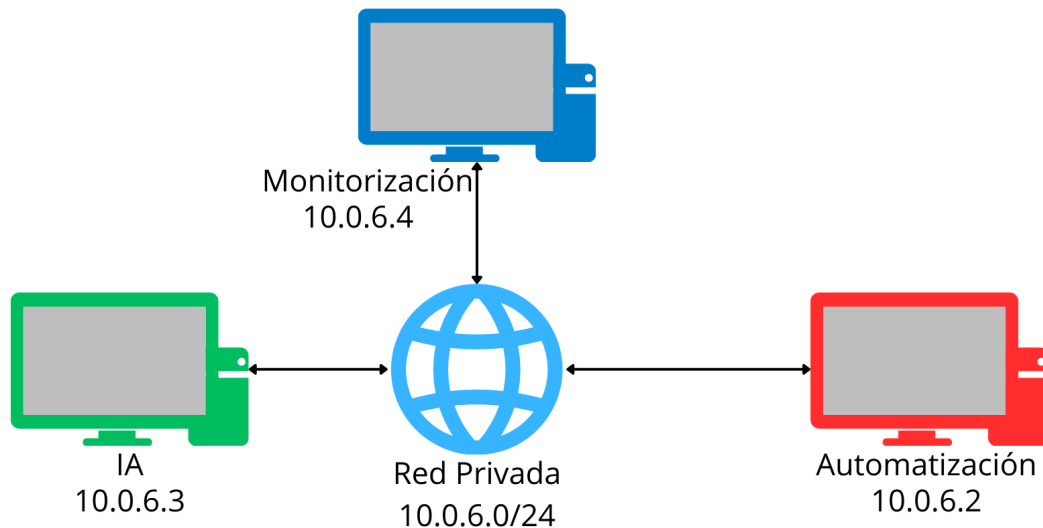
En esta práctica, se han usado los servicios de Google Cloud para crear tanto las máquinas virtuales que se usan para el sistema, como la red que las conecta entre sí.

Lo primero que hay que hacer en Google Cloud es crear un proyecto en el cual se puedan crear todos los elementos que sean necesarios. Una vez se crea el proyecto, es hora de preparar la red interna que conectará las máquinas virtuales, así como la/s subred/es (en este caso solo una), y las reglas del Firewall.

Esta red será denominada `vpc-proyecto-ecommerce`, y su única subred es denominada `subred-europa`, el direccionamiento IP de esta subred será `10.0.6.0/24`. Teniendo en cuenta que una de las máquinas virtuales actuará como un servidor web, y de que será necesaria una conexión SSH con todas ellas para poder llevar a cabo la práctica, se habilitarán los puertos 80 y 22 de TCP en el Firewall para las direcciones IPs no pertenecientes a la red interna, mientras que se permitirá el uso de todos los puertos UDP y TCP entre las máquinas de la red.

Debido a que Google Cloud tiene servidores por todo el mundo, es necesario a la hora de crear tanto las instancias como la red, elegir dónde ponerlas, siendo preferible usar en todas ellas una misma región para la disminución del coste a la hora de que las instancias tengan que comunicarse entre sí. La región escogida, tanto para la subred y para las máquinas virtuales, ha sido `europa-southwest1`, que pertenece a los servidores de Google Cloud ubicados en Madrid.

Las máquinas virtuales creadas han sido 3, una de ellas dedicada a la automatización, la denominada `vm-auto`, que se encargará de la usar Ansible para crear instalar los requerimientos en las demás máquinas e iniciar los servicios instalados y creados. Otra de las máquinas será llamada `vm-db-ai`, esta máquina ha sido usada para entrenar al modelo de la IA, esta será la máquina que servirá como servidor web, y ejecutará la IA para encontrar posibles ataques. La tercera y última máquina, `vm-mon-mgt`, será la encargada de monitorizar la máquina que actúa como servidor web y lanzar un script que lance una nueva instancia que actúe como nuevo servidor web.



3 NECESIDADES DE GESTIÓN

Las áreas de gestión aplicadas fueron las siguientes:

- **Gestión de Anomalías:** Fue necesario implementar un motor de inteligencia artificial capaz de detectar específicamente ataques de saturación (**DoS**) y anomalías en el tamaño de los paquetes. Mediante el uso del algoritmo **Isolation Forest**, el sistema aprende los tiempos de respuesta y volúmenes de datos normales, marcando cualquier desviación drástica del entrenamiento como una amenaza potencial.
- **Gestión de Control:** La **IA** por sí sola es un sensor pasivo que solo alerta. Necesitamos un agente activo que tenga la autoridad para interpretar esa alerta y ordenar la ejecución inmediata de contramedidas (**autoescalado**) en la infraestructura, cerrando así el ciclo de defensa sin intervención de una persona.
- **Automatización:** Es importante que cada máquina pueda funcionar de manera autónoma, sin necesidad de que haya una parte humana que ejecute los programas cuando estos se cierran por algún error o cada vez que se enciende la máquina de nuevo.

4 PLANIFICACIÓN

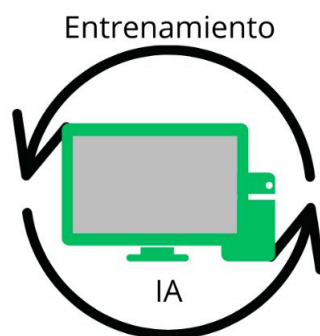
<u>Área</u>	<u>Tareas a realizar</u>	
Gestión de Anomalías	Elegir herramientas de ataques para la creación de códigos para entrenar al modelo de inteligencia artificial. Adaptar el código a la lectura de logs en tiempo real para detectar alertas.	
Gestión de Control	Configuración del Agente SNMP creando una OID dinámica para emplear la MIB. Archivo puente para leer alertas. Archivo python para hacer de panel de control.	
Automatización	Crear los servicios que ejecutan de manera automática los programas de detección con IA y de monitorización en las máquinas y los inicia en ambas con Ansible.	
Autoescalado	Fichero para lanzamiento de nueva instancia y modificación del archivo de panel de control para ejecutar remotamente el autoescalado desde otro host cuando la IA sufra un ataque .	

* Importante destacar que en todo momento todo se trabajó en conjunto de manera controlada y precisa, pero se dividió en roles para un mayor entendimiento de la memoria.

5 SOLUCIÓN DE GESTIÓN

A continuación mostraremos la solución detallada y ordenada de los pasos y orden que seguimos en equipo para realizar el proyecto separadas e introducidas por las distintas partes de gestión

5.1 ÁREA DE GESTIÓN DE ANOMALÍAS:



Nos encontrábamos ante una idea amplia, en la cual teníamos la idea clara de que, para detectar anomalías en el tráfico manejada por una inteligencia artificial, necesitábamos poder tener una IA que aprendiera por su cuenta cuando detectar logs anómalos o simplemente sospechosos. Por eso necesitábamos aplicar un modelo de machine learning.

Cuando nos documentamos, en un primer momento, queríamos entrenar a nuestra IA con ataques reales, por ello miramos conseguir dataset por internet con ataques inyectados, en nuestro caso al querer analizar logs de apache nos encontramos con "Apache Access Log Dataset" pero nos dio muchos problemas por dos simples razones:

- 1) El tamaño del archivo era demasiado grande por lo que nuestro código `entrenar_modelo.py` (que se explicará más adelante) colapsaba
- 2) las columnas y filas al no estar muy bien definidas nuestro gestor de anomalías no conseguía aprender por lo que a la hora de juntar todo el proyecto sería un auténtico caos.

Por eso mismo nos decantamos por crear un archivo `.py` llamado `generar_datos.py` que nos creará un conjunto de datos perfectamente estructurados para poder lograr el objetivo principal.

Para hacer este script, nos apoyamos en dos librerías fundamentales de Python que nos ayudarán mucho en tener unos datos ordenados y bien estructurados estas fueron:

NumPy: Esta librería era fundamental para generar datos aleatorios que sigan una distribución normal. Elegimos esta distribución porque, tal y como estudiamos en ingeniería de tráfico, nos permitirá modelar el comportamiento estadístico habitual de una red estable y con esto conseguimos distinguir matemáticamente lo que es un comportamiento estándar de una anomalía.

Pandas: Como la IA debe de aprender de nuestros datos necesitábamos asegurarnos que estos fueran ordenados y así nos permitiría utilizar un Data Frames de esos datos y exportarlos limpiamente a un archivo CSV.

A continuación se explican las líneas de código importantes a destacar por lo comentado anteriormente.

Como hemos dicho antes necesitábamos simular el comportamiento de una red real. El tráfico nunca es constante, por lo que utilizamos una distribución normal para generar los tiempos de respuesta y el tamaño de los paquetes. Esto nos creará un ruido natural aceptable, definiendo así lo que la IA debe considerar como tráfico normal

```
data = {  
    #Hacemos el tiempo que tarda en responder (segundos)  
    'tiempo_respuesta' : np.random.normal (loc = 0.5, scale = 0.1, size = 1000),  
  
    #Tamaño del paquete de datos (Bytes)  
    'tamano_paquete' : np.random.normal(loc = 500, scale = 100, size = 1000),  
  
    #Vamos a distinguir 0 si es trafico normal y 1 si es un ataque 10% ataques  
    'es_ataque' : np.random.choice([0, 1], 1000, p = [0.9, 0.1])  
}
```

Definido el comportamiento normal, establecimos que solo un 10% del tráfico serían ataques (para comprobar que funcione).

Sin embargo, para que el algoritmo de Machine Learning pudiera detectar estos ataques necesitábamos darle unas pequeñas pistas para que la IA pueda diferenciarlos matemáticamente.

Por eso aquí es donde implementamos dos ataques básicos conocidos en ciberseguridad uno son los ataques de latencia (DoS) y el otro básico conocido es la inyección de payloads pesados. por eso tuvimos que alterar artificialmente sus valores para convertirlos en valores atípicos claros:

```
# Para que el algoritmo de Machine Learning detecte los ataques, estos deben  
# diferenciarse matemáticamente del tráfico normal, DARLE UNA PSITA CLARA  
  
# 1) Podemos hacer una simulacion de ataques de Denegacion de Servicio que  
# bloqueen el servidor que provoquen una latencia alta  
  
df.loc[df['es_ataque'] == 1, 'tiempo_respuesta'] += 2.0 # consideramos latencia alta 2 seg  
  
# 2) Podemos hacer una simulacion de Payload Malicioso donde el atacante envia scripts  
# o consultas largas dentro del paquete.  
  
df.loc[df['es_ataque'] == 1, 'tamano_paquete'] += 1000
```

Gracias a este script, conseguimos exportar un archivo `datos_entrenamiento.csv` limpio, donde las anomalías no son errores de formato, sino patrones matemáticos intencionados que nuestra IA podrá aprender a identificar.

```
tiempo_respuesta,tamano_paquete,es_ataque
0.5496714153011233,639.9355436586002,0
0.4861735698828815,592.4633682912769,0
0.5647688538100692,505.9630369920174,0
0.6523029856408026,435.3063222294426,0
0.4765846625276664,569.822331361359,0
2.4765863043050818,1539.348538542175,1
0.6579212815507391,589.5193220027733,0
```

Con el dataset ya generado, el siguiente paso es poder entrenar a nuestro modelo, para ello se creó `entrenar_modelo.py` que creará el cerebro de nuestra Inteligencia Artificial.

Para poder realizar esta tarea, se necesitaron dos nuevas librerías.

Scikit-learn: Es la biblioteca estándar para algoritmos de aprendizaje automático.

Joblib: Aunque no haga una función en el código es vital, porque esto nos ha permitido guardar el modelo entrenado en un archivo físico para no tener que entrenarlo cada vez que iniciemos el sistema de detección que se explicara en la siguiente parte.

Hemos elegido el algoritmo Isolation Forest tras una larga búsqueda sobre librerías relacionadas con la ciberseguridad, ya que a diferencia de los antivirus tradicionales que buscan patrones de virus ya conocidos, el Isolation Forest es un algoritmo de detección de anomalías no supervisado. que es justo lo que necesitamos porque así simplemente aprende cómo es el tráfico normal y asume que cualquiera que se desvíe es una amenaza potencial.

Ya que lo nuestro es una simulación básica a nivel académico es perfecto para ver que hay ataques a nada que se pase ya que si lo ponemos más estricto y con ataques reales google cloud podía detectarlos y cerrar el servicio.

seleccionamos únicamente las columnas que aportan valor estadístico: el tiempo de respuesta y el tamaño del paquete

```
# Leemos el dataset generado en la fase anterior.
df = pd.read_csv('datos_entrenamiento.csv')
```

La IA aprenderá la correlación entre Tiempo y Tamaño

```
# La IA aprenderá la correlación entre Tiempo y Tamaño.
datos_aprendizaje = df[['tiempo_respuesta', 'tamano_paquete']]
```

Como sabemos por la fase de generación de datos que aproximadamente el 10% del tráfico eran ataques simulados realizamos un IsolationForest ajustando este valor al 10% para calibrar la sensibilidad del modelo y después hacemos que el modelo analice la geometría de los datos y que pueda trazar una frontera entre normal y anómalo.

```
modelo = IsolationForest(contamination=0.1, random_state=42)
|
# 3. ENTRENAMIENTO (FITTING)
# El modelo analiza la geometría de los datos y traza una frontera entre "Normal" y "Anómalo".
modelo.fit(datos_aprendizaje)
```

Finalmente guardamos el cerebro entrenado en un archivo binario .pkl usando el joblib mencionado al principio.

```
# Esto permite usar la IA en el script de detección sin tener que re-entrenar cada vez.
archivo_modelo = 'cerebro_ia.pkl'
joblib.dump(modelo, archivo_modelo)
```

El resultado de este script es el archivo cerebro_ia.pkl, un modelo matemático encapsulado y listo para ser desplegado en producción.

Finalmente para que podamos poner a prueba a nuestro modelo necesitamos crear un script para comprobar si efectivamente estamos detectando las anomalías por eso creamos el .py llamado detectar_en_vivo_old.py “old” porque habra que cambiarlo después para detectar tráfico real que nos mande la maquina 3.

Esto que vamos a hacer se conoce como Mocking, porque para que la recepción de los logs reales del servidor Apache que nos mandara la maquina3 y el agente SNMP que será responsabilidad de la maquina 2 se estaban desarrollando en paralelo, necesitábamos un entorno de pruebas independiente. No podíamos esperar a tener el servidor web montado para probar si nuestra IA era capaz de activar las alertas correctamente.

Una vez tengamos establecida dicha parte, en el apartado “ 5.3 CONEXIÓN ENTRE GESTIÓN DE ANOMALÍAS Y GESTIÓN DE CONTROL” se comentará todo este proceso. Pero de primeras, necesitamos crear un archivo estado_seguridad.txt que cambie en el tiempo según detecte alarmas (escriba 1 en el archivo) o no (escriba 0) para que la maquina 2 lo lea via SNMP y pueda monitorizar, pero se insiste en que en ese punto de comenta todo a la perfección de manera clara, ordenada y precisa.

Por eso configuramos el canal de comunicación definiendo el protocolo de comunicación con el agente SNMP. Como son procesos independientes, utilizamos un archivo de texto /tmp/estado_seguridad.txt

```
# Contenido: "1" (Alerta/Ataque) | "0" (Normal).

ARCHIVO_ALERTA = "/tmp/estado_seguridad.txt"
```

Después a la hora de cargar el modelo es importante destacar que en este punto todavía no estamos leyendo tráfico real. En su lugar, generamos datos matemáticos sobre la marcha e inyectamos ataques deliberados para forzar al sistema a reaccionar. Esto nos permite comprobar si la IA "dispara" la alerta cuando debe según lo aprendido

```
# --- MEJORA TÉCNICA ---
# Usamos abs() para evitar tiempos negativos (imposible en física).
# Usamos desviación 0.1 y 100 para simular una red estable (como en el entrenamiento).
tiempo_simulado = abs(np.random.normal(0.5, 0.1))
tamano_simulado = abs(np.random.normal(500, 100))

# TRUCO: Inyectamos un ataque el 10% de las veces para probar que la alerta roja funciona
if np.random.random() < 0.1:
    tiempo_simulado += 3.0 # Latencia muy alta
    tamano_simulado += 2000 # Paquete muy pesado

dato_real = [[tiempo_simulado, tamano_simulado]]

# 2. PREGUNTAMOS A LA IA
prediccion = modelo.predict(dato_real)
# En IsolationForest: -1 es anomalía (ataque), 1 es normal
es_ataque = True if prediccion[0] == -1 else False
```

Como se puede ver en la imagen en el mismo proceso preguntamos a la ia y cada dato, que ha sido simulado, pasa por el modelo IsolationForest. Si el modelo de aprendizaje nos devuelve un -1 lo detecta como anomalía y en el script escribe inmediatamente un "1" en el archivo de texto compartido. Si es tráfico normal, escribe un "0".

```
# 3. AVISAMOS A Monitorizacion (Escribimos en el archivo)
with open(ARCHIVO_ALERTA, "w") as f:
    if es_ataque:
        f.write("1")
    else:
        f.write("0")

if es_ataque:
    print(f"\033[91m🚨 ¡ATAQUE DETECTADO! Tiempo: {tiempo_simulado:.2f}s | Peso: {tamano_real:.0f}B\033[0m")
    # El archivo ya tiene un "1", así que nos quedamos quietos 1.5s para que Monitorizacion lo lea
    time.sleep(2)
else:
    print(f"\033[92m✅ Tráfico normal. Tiempo: {tiempo_simulado:.2f}s | Peso: {tamano_real:.0f}B\033[0m")
    time.sleep(1)
```

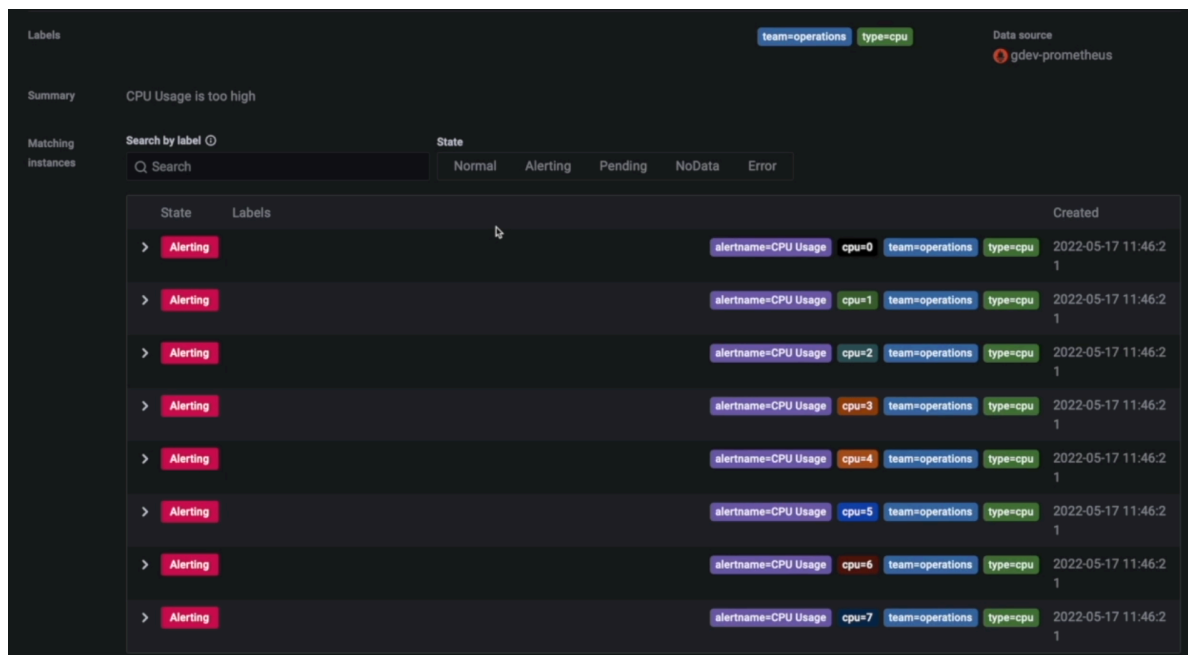
De esta forma, aunque el tráfico sea simulado, la activación de la alerta es real. El script leerá este archivo y, si encuentra un "1" realizará la monitorización correspondiente.

La prueba local quedaría de la siguiente forma:

```
montesponceantalban@vm-db-ai:~/Trabajo-GAR/modulo_ia$ python3 detectar_en_vivo.py
SISTEMA DE DEFENSA IA INICIADO
Escribiendo alertas para Adrián en: /tmp/estado_seguridad.txt
(Pulsa Ctrl+C para detener)
[✓]Tráfico normal. Tiempo: 0.59s | Peso: 460B
[✗]ATAQUE DETECTADO! Tiempo: 3.54s | Peso: 2531B
[✗]ATAQUE DETECTADO! Tiempo: 3.60s | Peso: 2477B
[✓]Tráfico normal. Tiempo: 0.48s | Peso: 500B
[✓]Tráfico normal. Tiempo: 0.55s | Peso: 337B
[✓]Tráfico normal. Tiempo: 0.49s | Peso: 584B
[✓]Tráfico normal. Tiempo: 0.61s | Peso: 543B
[✓]Tráfico normal. Tiempo: 0.58s | Peso: 441B
[✓]Tráfico normal. Tiempo: 0.27s | Peso: 451B
[✗]ATAQUE DETECTADO! Tiempo: 3.35s | Peso: 2459B
[✓]Tráfico normal. Tiempo: 0.44s | Peso: 482B
[✓]Tráfico normal. Tiempo: 0.39s | Peso: 373B
[✓]Tráfico normal. Tiempo: 0.62s | Peso: 650B
[✓]Tráfico normal. Tiempo: 0.41s | Peso: 553B
[✓]Tráfico normal. Tiempo: 0.46s | Peso: 527B
[✓]Tráfico normal. Tiempo: 0.50s | Peso: 550B
[✓]Tráfico normal. Tiempo: 0.34s | Peso: 619B
[✓]Tráfico normal. Tiempo: 0.55s | Peso: 623B
[✓]Tráfico normal. Tiempo: 0.52s | Peso: 472B
[✗]ATAQUE DETECTADO! Tiempo: 3.35s | Peso: 2400B
[✓]Tráfico normal. Tiempo: 0.67s | Peso: 648B
[✓]Tráfico normal. Tiempo: 0.50s | Peso: 459B
[✓]Tráfico normal. Tiempo: 0.38s | Peso: 380B
[✓]Tráfico normal. Tiempo: 0.37s | Peso: 425B
[✓]Tráfico normal. Tiempo: 0.41s | Peso: 545B
[✓]Tráfico normal. Tiempo: 0.44s | Peso: 502B
[✓]Tráfico normal. Tiempo: 0.48s | Peso: 677B
[✓]Tráfico normal. Tiempo: 0.40s | Peso: 558B
[✓]Tráfico normal. Tiempo: 0.68s | Peso: 431B
[✓]Tráfico normal. Tiempo: 0.62s | Peso: 484B
^C
Vigilancia detenida correctamente.
montesponceantalban@vm-db-ai:~/Trabajo-GAR/modulo_ia$
```

5.2 ÁREA DE GESTIÓN DE CONTROL(MONITORIZACIÓN)

Ejemplo de monitorización de alertas en Grafana como los vistos en el laboratorio



Antes de abordar la integración con el módulo de Inteligencia Artificial, fue necesario construir una base de monitorización capaz de centralizar el estado de la infraestructura. Nuestro objetivo principal era desarrollar un Nodo Gestor (**Manager**) que no dependiera de herramientas comerciales cerradas, sino que nos permitiera una personalización total del protocolo de alertas.

Tras analizar las necesidades del proyecto (sencillez y compatibilidad universal), descartamos el uso de agentes pesados y optamos por el estándar **SNMP** (**Simple Network Management Protocol**) en su versión 2c.

Definimos una arquitectura **Cliente-Servidor** clásica:

- **El Gestor (VM monitorización):** Actuaría como el panel central, responsable de lanzar las peticiones de estado.
- **El Agente (VM de IA):** Actuaría como el servidor pasivo que expone los datos.

Para preparar el entorno de gestión, diseñamos una máquina virtual dedicada (**vm-mon-mgt**) e instalamos las herramientas base del paquete **net-snmp**, incluyendo **snmpwalk** y **snmpget**, así como las librerías necesarias para la traducción de **MIBs** (**snmp-mibs-downloader**).

```
sudo apt-get update
```

```
sudo apt-get install snmp-mibs-downloader -y
```

Instalamos/actualizamos **SNMP** en la máquina **vm-mon-mgt** para poder hacer pruebas localmente antes de conectar con la máquina **vm-db-ai** posteriormente se deberá de hacer también en dicha máquina:

```
sudo apt-get update && sudo apt-get install snmp snmpd
```

En el siguiente apartado se indica el contenido y la configuración de los dos archivos necesarios para la conexión **snmp** entre la máquina de monitorización y la de **IA**(**snmpd.conf** para configuración snmp y **monitor_ia.sh** para hacer de puente y leer las alertas entre las **MV**).

Cada vez que se modifique el archivo de configuración **snmp** en cualquiera de las dos máquinas se deberá de reiniciar el servicio:

```
sudo service snmpd restart
```

Por otro lado, en la configuración del cliente **SNMP**, **Ubuntu** trae una línea que bloquea el uso de **MIBs** por defecto. Fue necesario comentarla:

En `/etc/snmp/snmp.conf`

```
# As the snmp packages come without MIB files due to license reasons, loading
# of MIBs is disabled by default. If you added the MIBs you can reenale
# loading them by commenting out the following line.
#mibs :
#
# If you want to globally change where snmp libraries, commands and daemons
# look for MIBS, change the line below. Note you can set this for individual
# tools with the -M option or MIBDIRS environment variable.
#
# mibdirs /usr/share/snmp/mibs:/usr/share/snmp/mibs/iana:/usr/share/snmp/mibs/ietf
```

De igual forma, tanto para poder probar el funcionamiento localmente como en la máquina de **IA**, hubo que habilitar permisos al archivo puente en ambas máquinas:

```
sudo chmod +x /usr/local/bin/monitor_ia.sh
```

Desarrollo del Dashboard de Control (El Algoritmo de Sondeo)

Dado que necesitábamos reaccionar ante una métrica no habitual (la predicción de un ataque), no podíamos usar un software de monitorización convencional. Por ello, desarrollamos nuestro propio software de gestión en **Python**, al que llamamos `dashboard_monitor.py`.

El script fue diseñado siguiendo una lógica de bucle infinito de sondeo, estructurado de la siguiente manera:

1. **Definición del Objetivo:** Configuramos variables estáticas para definir la **IP** objetivo y la Comunidad **SNMP (public)**, permitiendo cambiar el objetivo fácilmente sin tocar el código lógico.
2. **Ejecución de Comandos del Sistema:** Utilizamos la librería **subprocess** de **Python** para invocar directamente la herramienta **snmpwalk** del sistema operativo. Esto nos permitió capturar la salida del protocolo **SNMP**.
3. **Filtrado y Limpieza:** La respuesta de **SNMP** suele contener mucha "basura" (tipos de datos, **OIDs**, comillas...). Implementamos una función de limpieza **.decode().strip()** para aislar el dato puro que nos interesaba: un simple "0" o un "1".

Adicionalmente para evitar errores si `snmp-mibs-downloader` -y no funcionaba o no estaba instalado se podría directamente haber usado la **OID** en lugar de acceder a ella dinámicamente:

Usamos la OID numérica directa para evitar errores de MIBs faltantes

```
OID_NUMERICA = ".1.3.6.1.4.1.8072.1.3.2.4.1.2.9.101.115.116.97.100.111.95.105.97"
```

```
comando = [
```

```
    "snmpwalk", "-v2c", "-c", COMUNIDAD, "-Oqv",
```

```
    TARGET_IP, OID_NUMERICA
```

```
]
```

snmpwalk recorre por una rama del árbol de información.

-v2c versión de snmp basada en comunidades, ligera y rápida.

-c El Selector de Clave, **Community** (Comunidad).

COMUNIDAD La Variable, es una variable (que vale **public**). Debe coincidir con el **rocommunity** de la máquina de **IA**.

-Oqv Son opciones de salida (**Output**) combinadas:

- **-O**: Activa las opciones de formateo de salida.
- **q (Quick/Rápido)**: Elimina la información sobrante como el tipo de variable.
 - Sin **-q**: INTEGER: 1
 - Con **-q**: 1
- **v (Value/Valor)**: Elimina la **OID** (la dirección numérica). Muestra solo el valor.
 - Sin **-v**: iso.3.6.1.4.1.8072... = 1
 - Con **-v**: 1

```
# --- CONFIGURACIÓN ---
# IP MAQUINA IA
# Si queremos probar localmente contra nosotros mismos, usamos 'localhost'
TARGET_IP = "localhost"
COMUNIDAD = "public"

def consultar_estado_ia():
    try:
        # Comando snmpwalk para buscar la extensión que definimos como 'estado_ia'
        # nsExtendOutputFull es la rama estándar donde SNMP guarda las salidas de scripts
        comando = [
            "snmpwalk", "-v2c", "-c", COMUNIDAD, "-Oqv",
            TARGET_IP, "nsExtendOutputFull"
        ]

        # Ejecutamos el comando y limpiamos la salida
        resultado = subprocess.check_output(comando, stderr=subprocess.DEVNULL).decode().strip()
```

Lógica de Interpretación de Estados

Antes de tener la **IA** real conectada, definimos una tabla de verdad para que nuestro monitor supiera cómo interpretar los datos que recibiría en el futuro. Programamos el sistema para reconocer tres estados posibles:

- **ESTADO SEGURO (Valor 0):** El sistema confirma la conectividad y la ausencia de amenazas.
- **ESTADO PELIGRO (Valor 1):** El sistema detecta una anomalía crítica. Aquí preparamos el código para lanzar alertas visuales (**print** con emojis de alerta 🚨) y dejar el hueco para futuras acciones automáticas.

* Para poner emojis en Windows 10 u 11 empleamos la combinación de teclas “windows + .”

- **ESTADO ERROR/DESCONOCIDO:** Implementamos un manejo de excepciones (**try-except**) para detectar si el agente remoto estaba caído o inalcanzable (**Timeouts**), diferenciando así un "sistema seguro" de un "sistema apagado".

```
def consultar_estado_ia():
    try:
        # Comando snmpwalk para buscar la extensión que definimos como 'estado_ia'
        # nsExtendOutputFull es la rama estándar donde SNMP guarda las salidas de scripts
        comando = [
            "snmpwalk", "-v2c", "-c", COMUNIDAD, "-Oqv",
            TARGET_IP, "nsExtendOutputFull"
        ]

        # Ejecutamos el comando y limpiamos la salida
        resultado = subprocess.check_output(comando, stderr=subprocess.DEVNULL).decode().strip()

        # Buscamos nuestra etiqueta específica si hay muchas cosas
        # Buscamos el valor.
        if "1" in resultado.split("\n")[-1]:
            return "PELIGRO"
        elif "0" in resultado:
            return "SEGURO"
        else:
            return "DESCONOCIDO"

    except subprocess.CalledProcessError:
        return "ERROR_CONEXION"
```

4. Pruebas Unitarias en Local

Antes de intentar la conexión con la máquina remota, validamos el funcionamiento del **Dashboard** realizando pruebas de **Loopback**.

Configuramos un agente **SNMP** local en la propia máquina de monitorización (**localhost**), creamos un archivo local cuyo valor alternamos entre 0 o 1 manualmente y simulamos a mano los valores de la **MIB**. Esto nos permitió depurar el código de **Python** y ajustar la frecuencia de refresco a 1 o 2 segundos, para más adelante asegurarnos de que el script sea capaz de detectar cambios de estado en tiempo real sin saturar la **CPU** de la máquina de gestión.

Frecuencia con la que leemos el archivo(más tarde se cambió a 1 segundo)

```
time.sleep(2) # Preguntar cada 2 segundos
```

Muestra de funcionamiento cuando el archivo local está a '0'

```
ayipeton@vm-mon-mgt:~/Trabajo-GAR$ python3 dashboard_monitor.py
--- INICIANDO MONITORIZACIÓN DE IA EN 10.0.6.3 ---
Esperando datos del Agente SNMP...
[12:08:09] ✅ Sistema Estable...
```

Muestra de funcionamiento cuando el archivo local está a '1'

```
ayipeton@vm-mon-mgt:~/Trabajo-GAR$ python3 dashboard_monitor.py
--- INICIANDO MONITORIZACIÓN DE IA EN 10.0.6.3 ---
Esperando datos del Agente SNMP...
[19:23:54] 🚨 ALERTA CRÍTICA: ¡La IA ha detectado una amenaza! 🚨
[19:23:55] 🚨 ALERTA CRÍTICA: ¡La IA ha detectado una amenaza! 🚨
[19:23:56] 🚨 ALERTA CRÍTICA: ¡La IA ha detectado una amenaza! 🚨
[19:23:57] 🚨 ALERTA CRÍTICA: ¡La IA ha detectado una amenaza! 🚨
[19:23:58] 🚨 ALERTA CRÍTICA: ¡La IA ha detectado una amenaza! 🚨
^CTraceback (most recent call last):nquila.
  File "/home/ayipeton/Trabajo-GAR/dashboard_monitor.py", line 53, in <module>
    time.sleep(1) # Preguntar cada 1 segundos
KeyboardInterrupt
ayipeton@vm-mon-mgt:~/Trabajo-GAR$
```

5.3 CONEXIÓN ENTRE GESTIÓN DE ANOMALÍAS Y GESTIÓN DE CONTROL:



La idea principal en esta parte del proyecto era que el Área de control pudiera leer en tiempo real la detección de alertas que se hacía en la gestión de anomalías, recordar que en esta parte hicimos que si la **IA** detectaba algo intrusivo o sospechoso que cumpliera con **DoS** o inyección de archivos grandes que pudieran bloquear la red automáticamente la parte gestora de anomalías escribía en el archivo `/tmp/estado_seguridad.txt` "1" (si era un ataque) , "0" (si todo estaba bien) para que la parte de control pudiera hacer la monitorización.

Una vez teníamos clara la idea principal y el objetivo había que realizar varios pasos importantes, los explicamos a continuación:

La parte de control necesitaba poder desplegar y configurar el demonio **snmpd** en la máquina virtual encargada de la **IA**. Por eso nos encontramos con el reto de que, por defecto, el agente **SNMP** escucha únicamente en la interfaz de localhost (**127.0.0.1**) por motivos de seguridad, pero lo que realmente necesitábamos es que la gestión de control pudiera acceder de manera continuada al archivo que crea la **IA** dado que trabajamos en google cloud con la siguiente red:

Instancias de VM					
Filtro Escribir el nombre o valor de la propiedad					
☐ Estado	Nombre ↑	Zona	Recomendaciones	En uso por	IP interna
☐ 	vm-auto	europa-southwest1-a			10.0.6.2 (nic0)
☐ 	vm-db-ai	europa-southwest1-a			10.0.6.3 (nic0)
☐ 	vm-mon-mgt	europa-southwest1-a			10.0.6.4 (nic0)

Teníamos que permitir que el nodo de monitorización pudiera realizar consultas remotas. Realizamos las siguientes modificaciones críticas en el archivo `/etc/snmp/snmpd.conf` conociendo la red que tenemos lo primero es:

Control de Acceso: Debemos definir una comunidad de lectura (**rocommunity**) específica para nuestra red interna de Google Cloud (**10.0.6.0/24**). Esto nos garantiza que solo nuestros servidores autorizados puedan consultar el estado de la **IA**, rechazando conexiones de **IPs** desconocidas.

Implementación de la "MIB Particular" del apartado 5.2:

Creamos nuestra propia variable gestionada porque no existe una **OID** estándar para "Estado de Amenaza **IA**", por eso tras el estudio de qué era lo que debíamos utilizar, empleamos la funcionalidad de extensión de Net-SNMP (**NET-SNMP-EXTEND-MIB**).

El flujo de datos diseñado es el siguiente:

1. Generación del Dato: El script de Python de la **IA** escribe su predicción (0 = Seguro, 1 = Amenaza) en un archivo temporal (`/tmp/estado_seguridad.txt`). Elegimos el directorio `/tmp` para evitar conflictos de permisos de lectura/escritura entre el usuario de la **IA** y el usuario del sistema **SNMP**.
2. Script Intermediario: Desarrollamos un script en Bash (`monitor_ia.sh`) ubicado en `/usr/local/bin/` que lee dicho archivo y devuelve el valor numérico por la salida estándar.

Archivo *monitor_ia.sh*

```
#!/bin/bash
# Este script lee el archivo que genera la IA de David
# Ruta acordada con David
ARCHIVO="/tmp/estado_seguridad.txt"

if [ -f "$ARCHIVO" ]; then
    cat "$ARCHIVO"
else
    echo "0" # Si no existe, asumimos que todo está bien por seguridad
fi
```

Inserción de datos en la MIB: Vinculamos este script al agente **SNMP** mediante la directiva “extend” en la configuración:

```
extend estado_ia /bin/bash /usr/local/bin/monitor_ia.sh
```

Archivo *snmpd.conf*

```
# --- CONFIGURACIÓN SNMP ---

# 1. ACCESO:
# Permitimos localhost para la prueba 1, y también añadimos la red para la prueba final
rocommunity public 127.0.0.1
rocommunity public 10.0.6.0/24

# 2. VINCULACIÓN:
#Apuntamos al archivo puente
extend estado_ia /bin/bash /usr/local/bin/monitor_ia.sh
```

Esta configuración genera dinámicamente una nueva rama en el árbol de **OIDs** (bajo **.1.3.6.1.4.1.8072.1.3.2...**), permitiendo que el estado de la **IA** sea consultable como si fuera una métrica propia del sistema (como la **CPU** o la **RAM**).

En el software de Python (*dashboard_monitor.py*) fue necesario cada segundo hacer un sondeo contra la **IP** del servidor de **IA** para poder monitorizar el estado de la alerta.

```
# --- CONFIGURACIÓN ---
# IP DE DAVID
# Si queremos probar localmente contra ti mismo, usamos 'localhost'
TARGET_IP = "10.0.6.3"
COMUNIDAD = "public"
USUARIO_DAVID = "montesponcementsalban"
N_INSTANCIA = 1
```

Durante las pruebas de integración sin los logs reales, detectamos un problema de frecuencia de muestreo:

Si la **IA** detectaba un ataque, al siguiente segundo venía tráfico estable y el archivo de *estado_seguridad.txt* se mantenía a ‘1’ durante 1 segundo o menos. Por eso para que saliera bien el proyecto, al ser una simulación, forzamos que cuando detectara una amenaza la parte gestora de anomalías mantuviera la amenaza en el archivo unos 4 segundos; y el tráfico normal 1 segundo.

De esta manera cuando se detecta una amenaza, el sistema congela el estado de alerta en el archivo de intercambio durante 4 segundos. Esto garantiza matemáticamente que el Gestor **SNMP**(que fue modificado para leer cada segundo) tenga tiempo suficiente para leer, procesar y alertar sobre la incidencia, asegurando una fiabilidad del **100%** en la detección de ataques.

Si bien en un entorno de producción de alta velocidad el procesamiento no debería detenerse, para este prototipo hemos introducido pausas deliberadas (`time.sleep`) tras cada detección. Lo cual cumple las funciones críticas para la simulación del proyecto:

La primera a destacar es que somos capaces de garantizar que el agente **SNMP** (que lee el archivo de texto de forma asíncrona) tenga el tiempo suficiente para procesar el cambio de estado de '0' a '1' antes de que se sobrescriba el archivo, de esta forma nos permite visualizar y verificar paso a paso la activación de las alarmas durante la demostración, evitando que la consola se sature de logs locales y facilitando la depuración del sistema.

Maquina IA

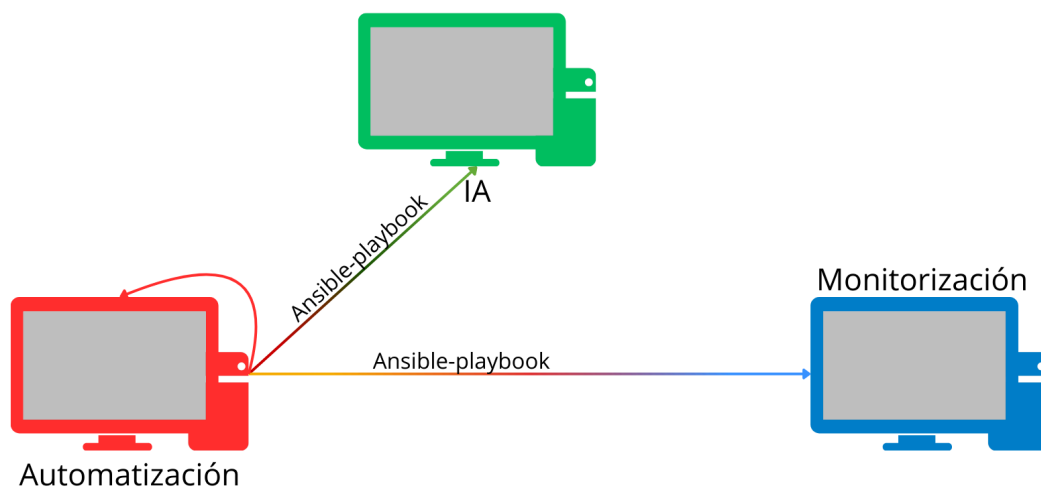
```
🚨;ATAQUE DETECTADO! Tiempo: 0.42s | Peso: 755B
✅Tráfico normal.      Tiempo: 0.60s | Peso: 730B
```

Maquina Monitorización

```
ayipeton@vm-mon-mgt:~/Trabajo-GAR$ python3 dashboard_monitor.py
--- INICIANDO MONITORIZACIÓN DE IA EN 10.0.6.3 ---
Esperando datos del Agente SNMP...
[16:52:13] 🚨ALERTA CRÍTICA: ;La IA ha detectado una amenaza! 🚨
[16:52:14] 🚨ALERTA CRÍTICA: ;La IA ha detectado una amenaza! 🚨
[16:52:15] 🚨ALERTA CRÍTICA: ;La IA ha detectado una amenaza! 🚨
[16:52:16] 🚨ALERTA CRÍTICA: ;La IA ha detectado una amenaza! 🚨
[16:52:17] 🚨ALERTA CRÍTICA: ;La IA ha detectado una amenaza! 🚨
[16:52:37] ✅Sistema Estable. IA tranquila.
```

* Más adelante con logs reales se volverá a cambiar este tiempo de estado por otros valores para que sea proporcionado a la gran cantidad de logs que hace Apache.

5.4 ÁREA DE AUTOMATIZACIÓN



Durante esta parte de la práctica, instalamos Ansible en la máquina de automatización.

En Ansible, hay que establecer un inventario con los Hosts que se van a cambiar, esto puede hacerse con un archivo en formato `.ini`, o usando el fichero ya existente `hosts`, que está en el directorio `/etc/hosts`.

```

[webservers]
# Máquina de David (Base de Datos + IA)
vm-db-ai ansible_host=10.0.6.3 ansible_user=montesponcemontalban ansible_become=yes ansible_become_method=sudo ansible_become_pass=''

[gestion]
# Máquina de Adrián (Monitorización)
vm-mon-mgt ansible_host=10.0.6.4 ansible_user=ayipeton ansible_become=yes ansible_become_method=sudo ansible_become_pass=''
  
```

Se establecen dos grupos llamados `webservers`, el cual tendrá la máquina que trabaja como servidor web, y `gestion`, que tendrá la máquina de monitorización. Cada máquina tiene de nombre de host de ansible el nombre que tiene establecido en Google Cloud, aunque no es necesario, y se asocia al host en cuestión mediante la dirección IP. Como cada participante se ha conectado con SSH a su máquina virtual correspondiente con su cuenta de Google, cada usuario era distinto, por lo que es necesario establecer que usuario debe utilizar ansible cuando vaya a conectarse con cada máquina. Una vez se entra en cada máquina con dichos usuarios, se le dice a ansible que debe convertirse en el usuario `root` (`sudo`).

Con el inventario creado, se pasa a crear el playbook de Ansible. En este se hacen tres 'plays' distintos.

- El primero actúa sobre el entorno del servidor web, y lo prepara instalando Apache, SNMPD para que el servidor web pueda ser monitorizado por el otro nodo. Copia también el fichero de configuración de SNMPD (`snmpd.conf`) en la instancia, y el script de monitorización que permite que la máquina de monitorización (`monitor_ia.sh`) tenga acceso al fichero que debe leer en el nodo de la IA.

Después se instalan las dependencias de python que el archivo de detección por IA pueda ejecutarse correctamente, esto se hace mediante PIP para poder elegir la versión preferida, ya que una de las librerías utilizadas para entrenar a la IA usaba la versión 2.0.0 de `numpy`, la cual no es la que se descarga con APT. Por último, se copia el script de detección de ataques en el servidor web en el directorio `/usr/local/bin`, junto al cerebro de la IA.

La última tarea que se hace en el servidor web es la de crear el servicio, esto se hace usando una plantilla de Ansible, fichero acabado en `.j2`, que al lanzar el playbook permitirá reconocer variables de Ansible y utilizarlas en el fichero final que es copiado en el destino.

- El segundo 'play', actúa sobre el host de monitorización, se asegura de que python y SNMP están instalados y activos en dicha máquina, y prepara el servicio de monitorización del dashboard del mismo modo que el servicio creado en el play anterior.
- El tercer y último 'play' se fija en la propia máquina de automatización, y lo único que hace es instalar el paquete de `apache2-utils`, para que se puedan hacer las pruebas de red usando el comando `ab` disponible en dicho paquete, que lanza varias peticiones HTTP de manera simultánea a un servidor web.

`gar-defense.service.j2`

```
[Unit]
Description=Sistema de Defensa IA (David)
After=network.target

[Service]
Type=simple
User=root
WorkingDirectory=/usr/local/bin
ExecStart=/usr/bin/python3 {{ ai_dir }}
Restart=always
RestartSec=5
Environment=PYTHONUNBUFFERED=1

[Install]
WantedBy=multi-user.target
```

```
[Unit]
Description=Dashboard de Monitorización SNMP (Adri)
After=network.target

[Service]
Type=simple
# Ajusta la ruta si decides ponerlo en otro sitio
ExecStart=/usr/bin/python3 {{ dashboard_dest }}
Restart=always
RestartSec=5
User=root
# Esto hace que los print() de Python salgan
# inmediatamente en el log
Environment=PYTHONUNBUFFERED=1

[Install]
WantedBy=multi-user.target
```

gar-monitor.service.j2

Como ya se ha explicado, para poder garantizar que los scripts de monitorización y de detección de ataques sean ejecutados de manera continua y automática al reiniciar las máquinas, se ha decidido crear unos servicios del sistema de linux.

Estos servicios se establecerán en el directorio `/etc/systemd/system/`, lo cual permitirá que Systemd; que es el sistema encargado de lanzar el resto de los servicios del sistema de Linux, y es el primer proceso en ser ejecutado^[1], pueda lanzarlos al iniciarse la instancia.

La estructura de los servicios se separa en 3 bloques funcionales:

1. Definición y dependencias ([Unit]):

- **Description:** No influye de ningún modo en el funcionamiento de los servicios, solo ofrece una descripción del funcionamiento del sistema.
- **After/Before:** elige un servicio objetivo que permite dar un nivel de prioridad a los servicios, es decir, si necesitas que un servicio se inicie antes que el que estás configurando; ya que este requiere el funcionamiento de dicho servicio, pones `After=service.target`, también se puede usar `Before=servicio.target`, para que este nuevo servicio se inicialice antes que el otro. En este caso, los servicios creados necesitan del servicio de red para poder funcionar y comunicarse entre sí, por lo que se utiliza la línea `After=network.target`.

2. Definición del servicio ([Service]): este bloque define cómo va a funcionar el servicio, declarando cosas como, qué usuario lo ejecuta, qué código debe ejecutar, etc.

- **Tipo:** establece como debe de funcionar el `ExecStart` a la hora de iniciar el servicio, en este caso, al usar `Type=simple`, se especifica que el proceso principal del servicio es `ExecStart`, y no un proceso hijo.
- **Usuario:** elige qué usuario es el encargado de ejecutar el proceso perteneciente al servicio, al escoger el usuario `root`, se permite que el proceso de monitorización acceda a las claves SSH necesarias para conectarse con las otras instancias.

- `ExecStart`: dice donde se encuentra el script de ejecución del servicio, en este caso, también se indica que el entorno de ejecución es `python3`.
- `Reiniciar`: al usar el valor `always`, se reinicia el proceso en caso de que este salga por algún error, de este modo el servicio nunca se parará, aunque haya fallos. El único modo de parar el servicio es al utilizar el comando `systemctl`. Se establece que el tiempo de reinicio es de 5 segundos.
- `Entorno`: se crea una variable de entorno, en este caso `PYTHONBUFFERED`, y le da el valor 1. Esta variable indica que los logs de salida (`print()`) de python no deben almacenarse en el `buffer`, permitiendo así observarlos a tiempo real en el diario del sistema (`journalctl`).

3. Integración de arranque ([Install]):

- `WantedBy`: Se trata de una lista de unidades bajo las que el servicio debería ser iniciado. En este caso, se escoge la unidad `multi-user.target`, lo que significa que el servicio será iniciado cuando se alcance un estado operativo multiusuario.

Un detalle importante a tener en cuenta en el playbook, es que al iniciar dos nuevos servicios, uno en cada máquina, estos servicios deben ser iniciados desde con Ansible desde el playbook. De no hacerse esto, será necesario usar el comando `systemctl` directamente en las instancias objetivo o reiniciar las máquinas virtuales para que Systemd pueda iniciarlas por su cuenta. Esto se debe a que Systemd solo puede lanzar los servicios bajo su cargo al iniciarse la máquina virtual.

Maquina Monitorización

```
ayipeton@vm-mon-mgt:~/Trabajo-GAR$ sudo systemctl status gar-monitor.service
● gar-monitor.service - Dashboard de Monitorización SNMP (Adri)
   Loaded: loaded (/etc/systemd/system/gar-monitor.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-01-10 12:09:07 UTC; 9min ago
     Main PID: 13564 (python3)
        Tasks: 1 (limit: 19180)
       Memory: 4.0M
          CPU: 12.068s
      CGroup: /system.slice/gar-monitor.service
              └─13564 /usr/bin/python3 /home/ayipeton/dashboard_monitor.py

Jan 10 12:09:07 vm-mon-mgt systemd[1]: Started Dashboard de Monitorización SNMP (Adri).
Jan 10 12:09:07 vm-mon-mgt python3[13564]: --- INICIANDO MONITORIZACIÓN DE IA EN 10.0.6.3 ---
Jan 10 12:09:07 vm-mon-mgt python3[13564]: Esperando datos del Agente SNMP...
```

Maquina IA

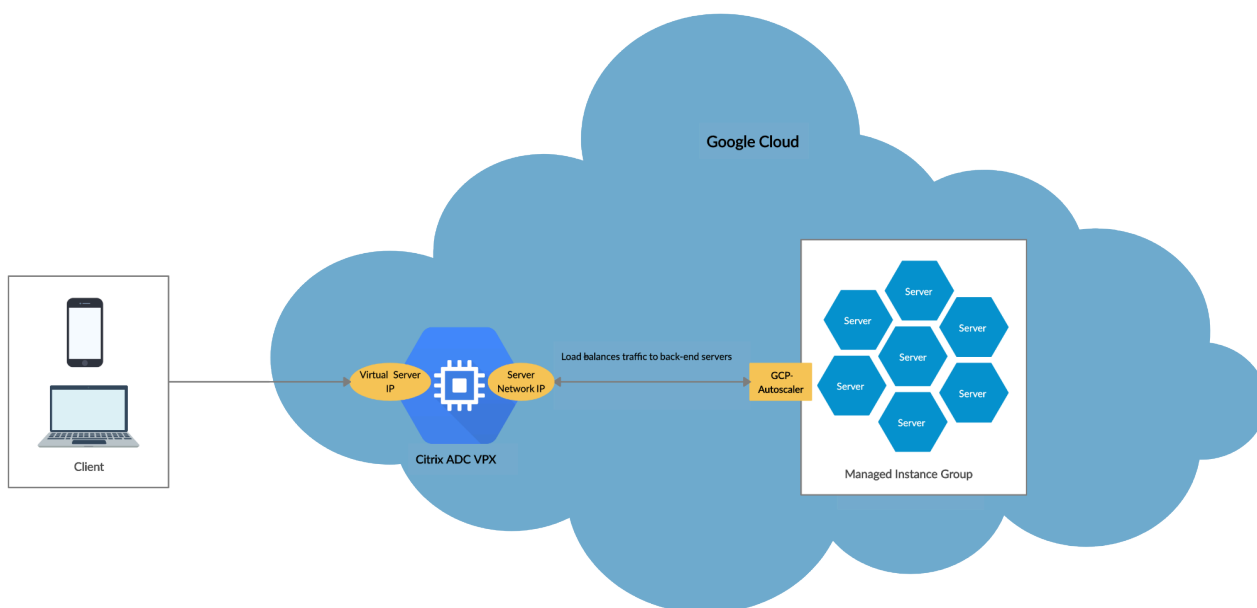
```

0montesponcemontalban@vm-db-ai:~/Trabajo-GAR/modulo_ia$ sudo rm /tmp/estado_seguridad.txt /tmp/historial_escalado.txt
montesponcemontalban@vm-db-ai:~/Trabajo-GAR/modulo_ia$ ls /tmp/
snap-private-tmp
ssh-XXXX64trcl
systemd-private-6fa0b8f89f124dcf98409ad203a5777c-apache2.service-Bj81bD
systemd-private-6fa0b8f89f124dcf98409ad203a5777c-chrony.service-14ZGmF
systemd-private-6fa0b8f89f124dcf98409ad203a5777c-systemd-logind.service-EE7zAL
systemd-private-6fa0b8f89f124dcf98409ad203a5777c-systemd-resolved.service-DBC50t
montesponcemontalban@vm-db-ai:~/Trabajo-GAR/modulo_ia$ sudo systemctl status gar-defense.service
● gar-defense.service - Sistema de Defensa IA (David)
   Loaded: loaded (/etc/systemd/system/gar-defense.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-01-10 12:18:22 UTC; 3min 33s ago
     Main PID: 13918 (python3)
        Tasks: 7 (limit: 19180)
      Memory: 68.0M
         CPU: 1.974s
       CGroup: /system.slice/gar-defense.service
              └─13918 /usr/bin/python3 /usr/local/bin/detectar_en_vivo.py

Jan 10 12:18:22 vm-db-ai systemd[1]: Started Sistema de Defensa IA (David).
montesponcemontalban@vm-db-ai:~/Trabajo-GAR/modulo_ia$ ls /tmp/
estado_seguridad.txt
snap-private-tmp
ssh-XXXX64trcl
systemd-private-6fa0b8f89f124dcf98409ad203a5777c-apache2.service-Bj81bD
systemd-private-6fa0b8f89f124dcf98409ad203a5777c-chrony.service-14ZGmF
systemd-private-6fa0b8f89f124dcf98409ad203a5777c-systemd-logind.service-EE7zAL
systemd-private-6fa0b8f89f124dcf98409ad203a5777c-systemd-resolved.service-DBC50t

```

5.5 AUTOESCALADO



Habíamos conseguido encontrar de manera automática amenazas y alertas y mostrar dichas alertas por pantalla una vez detectadas. Ahora llega el momento de actuar de algún modo ante estas amenazas. Aquí entra el papel del autoescalado con el objetivo de ante la llegada de un archivo muy grande o una latencia de respuesta elevada (como se configuró en el punto 5.1) generar una copia del estado de la instancia y activarla, destruyendo la original asumiendo que fue comprometida. De esta forma cerraremos el sistema dotándolo de capacidad de auto-reparación sin necesidad de intervención humana. Todo esto fue desarrollado de forma simulada y visual simplemente para probar como funciona en un caso poco realista.

Para implementar esta lógica de amenaza y respuesta, desarrollamos los siguientes componentes:

Diseño del Mecanismo de Disparo (Triggering)

En el nodo de monitorización, modificamos el Gestor **SNMP** (*dashboard_monitor.py*) para que dejara de ser solo un visualizador pasivo. Lo primero de todo fue modificar el archivo para que ya no mostrará por cada alerta que generaba la máquina de **IA** otras 3 simultáneas sino que se mostrará un mensaje indicando que la alerta sigue activa que también serviría en caso de que apareciera otra amenaza seguida.

```
# 3. FEEDBACK VISUAL (Para saber que sigue vivo)
if estado_actual == "SEGURO":
    print(f"[{fecha}] ✅ Sistema Estable...", end="\r")
elif estado_actual == "PELIGRO":
    print(f"[{fecha}] ⚠️ ATAQUE EN CURSO...", end="\r")
```

A continuación se generó una clave **ssh** en la máquina de monitorización, compartida con la máquina de **IA** y la de automatización para que el script de panel de monitorización pueda ejercer órdenes shell de manera remota sobre dichas instancias.

```
ayipeton@vm-mon-mgt:~/Trabajo-GAR$ ssh-keygen -t rsa # (Dale a Enter a todo)
Generating public/private rsa key pair.
Enter file in which to save the key (/home/ayipeton/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ayipeton/.ssh/id_rsa
Your public key has been saved in /home/ayipeton/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:JDQf8rUNtyYctnIRI3Czk6ZPm25e8r+sNlCryH4HXgY ayipeton@vm-mon-mgt
The key's randomart image is:
+---[RSA 3072]-----+
|      =. = O..      |
|      . * X X .      |
|      . X * +        |
|      =E+.o          |
|      . So .         |
|      ooo+           |
|      . o=*          |
|      o.=++         |
|      ..+ooo++       |
+-----[SHA256]-----+
```

```
opciones_ssh = [
    "-o", "StrictHostKeyChecking=no",
    "-o", "ConnectTimeout=5",
    "-i", "/home/ayipeton/.ssh/id_rsa"
]
```

En el código de Python, es necesario añadir estas opciones para las conexiones **SSH**, se tratan de opciones del propio comando de **SSH**. `-o` permite añadir opciones al comando, mientras que la opción `-i` permite elegir la clave **SSH** a utilizar, siendo esta la clave creada anteriormente.

En este caso, se especifican las opciones `StrictHostKeyChecking=no` y `ConnectTimeout=5`, la primera hace que al utilizar la conexión **SSH** no se haga la típica pregunta de que si se está seguro de querer conectarse, ya que queremos que se haga de manera automática por parte del servicio creado y que no sea necesaria la intervención humana. La segunda opción indica el tiempo máximo de espera para realizar la conexión, en el caso de tardar más de 5 segundos, se ve como un error.

Una vez ya la máquina de monitorización tiene conexión **ssh** sobre la máquina de **IA**, para que no lleguen nuevas alertas mientras se soluciona un problema, se configuró el archivo de control de tal forma que apague el servicio web de apache (`systemctl stop apache2`) para que no se pueda conectar nadie durante ese tiempo. Después, una vez que se hace el autoescalado y la nueva máquina está habilitada se reanuda el servicio web (`systemctl start apache2`) y volveríamos a recibir nuevos logs. En un entorno de producción real, este paso lanzaría instancias adicionales (**Auto-scaling**) mediante herramientas como **Ansible** o **Terraform**; en nuestra simulación, optamos por un reinicio limpio del servicio para demostrar la capacidad de recuperación automática del sistema.

Intentamos parar apache

```
try:
    cmd = ["ssh"] + opciones_ssh + ["montesponcemonalban@10.0.6.3", "sudo systemctl stop apache2"]
    subprocess.run(cmd, capture_output=True, text=True, check=True)
    print(f"[{fecha}] PARADO EL SERVICIO WEB")
except subprocess.CalledProcessError as e:
    print(f"[{fecha}] ✗ ERROR AL PARAR EL SERVICIO WEB: {e.stderr}")
```

Arrancamos de nuevo apache una vez está todo solucionado

```
try:
    cmd = ["ssh"] + opciones_ssh + ["montesponcemonalban@10.0.6.3", "sudo systemctl start apache2"]
    subprocess.run(cmd, capture_output=True, text=True, check=True)
    print(f"[{fecha}] REANUDADO EL SERVICIO WEB")
except subprocess.CalledProcessError as e:
    print(f"[{fecha}] ✗ ERROR AL REANUDAR EL SERVICIO WEB: {e.stderr}")
```

Ejecución del fichero de autoescalado

```
try:
    # Ejecutamos el comando y esperamos respuesta
    cmd = ["ssh"] + opciones_ssh + ["montesponcemonalban@10.0.6.3", f"python3 /usr/local/bin/autoscale.py {N_INSTANCIA}"]
    resultado = subprocess.run(cmd, capture_output=True, text=True, check=True)
    N_INSTANCIA = N_INSTANCIA + 1
    print(f"[{fecha}] RESPUESTA DE DAVID: {resultado.stdout.strip()}")
except subprocess.CalledProcessError as e:
    print(f"[{fecha}] ✗ ERROR AL ESCALAR: {e.stderr}")
```

Aunque el fichero de autoescalado está en la máquina de la **IA**, y este se ejecuta en la misma, en un entorno real, este fichero sería ejecutado en la máquina de automatización, puesto que este autoescalado eliminaría la instancia de la **IA** para crear una nueva. Al ejecutar cada fichero, se envía una variable que enumera las instancias a medida que se van lanzando, actualizando dicho número con cada ataque solucionado.

El fichero de autoescalado (`autoscale.py`), en este caso solo crea unos logs y los guarda en un fichero creado por el mismo en el directorio `/tmp`.

Adicionalmente para diferenciar la resolución de cada alerta, se enumeró cada una de ellas

```
TARGET_IP = "10.0.6.3"
COMUNIDAD = "public"
USUARIO_DAVID = "montesponcemontalban"
N_INSTANCIA = 1
```

Por último modificamos el archivo de `estado_seguridad.txt` con un '0' para que al volver a detectar ataques no salte una falsa alerta nada más reiniciar el sistema.

Ponemos a '0' el archivo de alerta

```
try:
    cmd = ["ssh"] + opciones_ssh + ["montesponcemontalban@10.0.6.3", "echo 0 | sudo tee /tmp/estado_seguridad.txt > /dev/null"]
    subprocess.run(cmd, capture_output=True, text=True, check=True)
except subprocess.CalledProcessError as e:
    print(f"[{fecha}] ✗ ERROR AL CAMBIAR EL ESTADO DEL LOG: {e.stderr}")
```

Respuesta del panel de control ante distintos ataques seguidos

```
ayipeton@vm-mon-mgt:~/Trabajo-GAR$ python3 dashboard_monitor.py
--- INICIANDO MONITORIZACIÓN DE IA EN 10.0.6.3 ---
Esperando datos del Agente SNMP...
[11:59:29] 🚨 ALERTA CRÍTICA: ¡La IA ha detectado una amenaza! 🚨
[11:59:29] 📡 Enviando orden de escalado a la máquina de David (10.0.6.3)...
[11:59:29] PARADO EL SERVICIO WEB
[11:59:29] RESPUESTA DE DAVID: !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[2026-01-10 11:59:30] RECIBIDA ORDEN DE ESCALADO DE EMERGENCIA
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

[2026-01-10 11:59:30] Contactando con API del Cloud Provider...
[2026-01-10 11:59:31] Desplegando nueva instancia 'web-server-rescue-1'...
[2026-01-10 11:59:33] Instancia iniciada y operativa.
[2026-01-10 11:59:33] Actualizando reglas del Balanceador de Carga...
[2026-01-10 11:59:34] Balanceador reconfigurado. Distribuyendo tráfico.

=====
[2026-01-10 11:59:34] PROTOCOLO DE DEFENSA COMPLETADO CON ÉXITO.
=====

[11:59:29] REANUDADO EL SERVICIO WEB
[12:03:42] 🚨 ALERTA CRÍTICA: ¡La IA ha detectado una amenaza! 🚨
[12:03:42] 📡 Enviando orden de escalado a la máquina de David (10.0.6.3)...
[12:03:42] PARADO EL SERVICIO WEB
[12:03:42] RESPUESTA DE DAVID: !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[2026-01-10 12:03:43] RECIBIDA ORDEN DE ESCALADO DE EMERGENCIA
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

[2026-01-10 12:03:43] Contactando con API del Cloud Provider...
[2026-01-10 12:03:44] Desplegando nueva instancia 'web-server-rescue-2'...
[2026-01-10 12:03:46] Instancia iniciada y operativa.
[2026-01-10 12:03:46] Actualizando reglas del Balanceador de Carga...
[2026-01-10 12:03:47] Balanceador reconfigurado. Distribuyendo tráfico.

=====
[2026-01-10 12:03:47] PROTOCOLO DE DEFENSA COMPLETADO CON ÉXITO.
=====

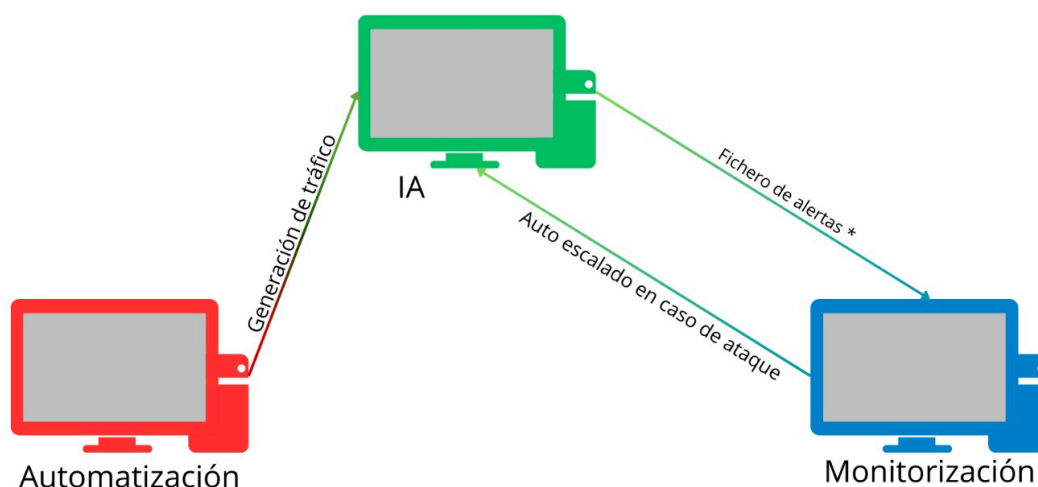
[12:03:42] REANUDADO EL SERVICIO WEB
[12:03:53] ⚠️ ATAQUE EN CURSO...
```

Respuesta del autoescalado almacenada en el historial de la máquina de IA

```

montesponcemontalban@vm-db-ai:~/Trabajo-GAR/modulo_ia$ sudo cat /tmp/historial_escalado.txt
[2026-01-10 11:56:08] RECIBIDA ORDEN DE ESCALADO DE EMERGENCIA
[2026-01-10 11:56:08] Contactando con API del Cloud Provider...
[2026-01-10 11:56:10] Desplegando nueva instancia 'web-server-rescue-1'...
[2026-01-10 11:56:12] Instancia iniciada y operativa.
[2026-01-10 11:56:12] Actualizando reglas del Balanceador de Carga...
[2026-01-10 11:56:13] Balanceador reconfigurado. Distribuyendo tráfico.
[2026-01-10 11:56:13] PROTOCOLO DE DEFENSA COMPLETADO CON ÉXITO.

```

5.6 PUESTA EN MARCHA DEL SISTEMA

Una vez todo está funcionando, es hora de enviar tráfico de prueba para poder observar cómo trabajan la **IA** y el gestor ante una situación en la que llega tráfico de manera continuada.

A diferencia de la versión anterior en la cual la **IA** se inventaba los datos, este código se conecta directamente al flujo de información del servidor web Apache leyendo el archivo `/var/log/apache2/access.log` en tiempo real.

Para hacer esta cuestión tuvimos que hacer una serie de pasos importantes a continuación, se presentarán y comentarán uno a uno mostrando evidencias del código para que se entienda mucho mejor.

Los logs de un servidor no son un archivo estático, crecen constantemente con cada visita. Para ello tuvimos que hacer una lectura continua de flujo de datos. Para esto hicimos uso del `tail -f` mediante una función generadora `yield` que emula el comportamiento de este comando de Linux. De esta forma la **IA** puede analizar el tráfico al instante ya que al ser un análisis en tiempo real no podíamos leer el archivo y cerrarlo.

Debíamos situarnos al final del archivo y esperar a que aparezcan nuevas líneas.

```
# COMO EL ARCHIVO CAMBIA CONSTANTEMENTE CON LOS LOGS DEBEMOS DE LEER LINEA A LINEA SEGUN
#ENTREN LOS DATOS
def seguir_log(archivo):
    archivo.seek(0, os.SEEK_END) #vamos al final del archivo
    while True:
        linea = archivo.readline()
        if not linea:
            time.sleep(0.1)
            continue
        yield linea
```

Tenemos que tener en cuenta que **Apache** genera cadenas de texto bastante “sucias” para una **IA** ya que estas contienen fechas, IPs, navegadores, etc., y el modelo matemático que habíamos implementado solo entiende números. Es por esto que fue necesario programar un **parsing**, con el fin de separar cada línea de texto que entra para aislar así el dato que nos interesa; que es el tamaño del paquete.

Durante la prueba nos encontramos con un problema real de los servidores Apache. Lo que sucedía era que cuando una respuesta es de 0 bytes, en lugar de poner un "0", Apache escribe un guión; por eso era necesario programar una excepción específica para convertir ese carácter en un valor numérico procesable y evitar que el sistema colapsara.

```
#REALIZAMOS LA LINEA DE DATOS
partes = linea.split(' ')

#Por si hay lineas vacias
if len(partes) < 3:
    continue

# El tamaño siempre es la ultima palabra de la linea de Apache
datos_servidor = partes[2].split()

#Si sale un guion el tamaño es 0
if len(datos_servidor) < 2:
    continue

dato_bytes_texto = datos_servidor[1]

if dato_bytes_texto == '-':
    tamaño_real = 0.0 # Corregido typo (ñ -> n) para que coincida abajo
else:
    tamaño_real = float(dato_bytes_texto)
```

Como queríamos mantener la idea del entrenamiento que hicimos para detectar ataques de **DoS**, y la configuración estándar de Apache no nos entregaba el tiempo de latencia con la precisión necesaria sin modificar el núcleo del servidor. Optamos por un enfoque híbrido, en el cual usamos el tamaño real del paquete (leído en tráfico real) combinado con una latencia simulada estable. De esta manera, nuestra **IA** tiene los dos parámetros que necesita para funcionar como bien estipulamos en el entrenamiento ([tiempo, tamaño]) y es capaz de detectar anomalías basándose principalmente en el peso de los datos transferidos.

```
# 2. GESTIÓN DEL TIEMPO (LATENCIA)
# Como Apache no nos da el tiempo sin config avanzada,
# simulamos un tiempo "estable" para que la IA no se confunda.
# PERO: Usamos el TAMAÑO REAL para detectar el ataque.

tiempo_simulado = abs(np.random.normal(0.5, 0.1))

dato_para_ai = [[tiempo_simulado, tamano_real]]

# 2. PREGUNTAMOS A LA IA
prediccion = modelo.predict(dato_para_ai)
```

Finalmente, el sistema cierra el ciclo de la misma forma que en el prototipo: si la **IA** detecta una anomalía en estos datos reales, escribe la alerta en el archivo compartido para que el agente **SNMP** ejecute la defensa.

```
montesponceantalban@vm-db-ai:~/Trabajo-GAR/modulo_ia$ python3 detectar_en_vivo.py
SISTEMA DE DEFENSA IA INICIADO
Escucando log reales de Rodri en: /var/log/apache2/access.log
Escribiendo alertas para Adrián en: /tmp/estado_seguridad.txt
(Pulsa Ctrl+C para detener)
🚨 ATAQUE DETECTADO! Tiempo: 0.44s | Peso: 50000B
🚨 ATAQUE DETECTADO! Tiempo: 0.49s | Peso: 50000B
🚨 ATAQUE DETECTADO! Tiempo: 0.40s | Peso: 50000B
🚨 ATAQUE DETECTADO! Tiempo: 0.52s | Peso: 50000B
🚨 ATAQUE DETECTADO! Tiempo: 0.39s | Peso: 50000B
🚨 ATAQUE DETECTADO! Tiempo: 0.46s | Peso: 50B
🚨 ATAQUE DETECTADO! Tiempo: 0.39s | Peso: 50B
✅ Tráfico normal. Tiempo: 0.38s | Peso: 500B
✅ Tráfico normal. Tiempo: 0.52s | Peso: 500B
✅ Tráfico normal. Tiempo: 0.46s | Peso: 500B
```

Las imágenes de la prueba siguiente se han hecho apagando los servicios creados y ejecutando los códigos con el comando `python3`.

La máquina de automatización en este caso funciona como emisor de tráfico, utilizando el comando `curl` para hacer varias peticiones. Primero se hacen peticiones **GET** normales al sitio web para obtener la página web principal. Después se tratará de subir al servidor un archivo pesado que la **IA** interprete como un intento de ataque.

La imagen siguiente muestra la salida de la detección con IA, se han hecho 3 peticiones normales y 2 intentos de subir el archivo pesado. El primero de los intentos no ha sido detectado como ataque, pero el segundo si ha hecho saltar la alarma de la IA, por lo que esta cambia el estado en el fichero al que accede el gestor.

```
montesponce montalban@vm-db-ai:~/Trabajo-GAR/modulo_ia$ python3 detectar_en_vivo.py
SISTEMA DE DEFENSA IA INICIADO
Escutando log reales de Rodri en: /var/log/apache2/access.log
Escribiendo alertas para Adrián en: /tmp/estado_seguridad.txt
(Pulsa Ctrl+C para detener)
✓Tráfico normal.      Tiempo: 0.35s | Peso: 729B
✓Tráfico normal.      Tiempo: 0.42s | Peso: 729B
✓Tráfico normal.      Tiempo: 0.58s | Peso: 729B
✓Tráfico normal.      Tiempo: 0.47s | Peso: 754B
🚨¡ATAQUE DETECTADO! Tiempo: 0.61s | Peso: 754B
```

Cuando el gestor se percató del cambio en el fichero, éste manda una orden al servidor web para parar el servicio de apache, ejecuta el código de autoescalado y cambia el estado del archivo vigilado a 0 para no volver a ver el error anterior antes de encender de nuevo el servicio de apache.

```
ayipeton@vm-mon-mgt:~/Trabajo-GAR$ python3 dashboard_monitor.py
--- INICIANDO MONITORIZACIÓN DE IA EN 10.0.6.3 ---
Esperando datos del Agente SNMP...
[11:56:07] 🚨ALERTA CRÍTICA: ¡La IA ha detectado una amenaza! 🚨
[11:56:07] 📡Enviando orden de escalado a la máquina de David (10.0.6.3)...
[11:56:07] PARADO EL SERVICIO WEB
[11:56:07] RESPUESTA DE DAVID: !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[2026-01-10 11:56:08] RECIBIDA ORDEN DE ESCALADO DE EMERGENCIA
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

[2026-01-10 11:56:08] Contactando con API del Cloud Provider...
[2026-01-10 11:56:10] Desplegando nueva instancia 'web-server-rescue-1'...
[2026-01-10 11:56:12] Instancia iniciada y operativa.
[2026-01-10 11:56:12] Actualizando reglas del Balanceador de Carga...
[2026-01-10 11:56:13] Balanceador reconfigurado. Distribuyendo tráfico.

=====
[2026-01-10 11:56:13] PROTOCOLO DE DEFENSA COMPLETADO CON ÉXITO.
=====
[11:56:07] REANUDADO EL SERVICIO WEB
[11:56:22] ✓Sistema Estable...No hay alertas
```

El panel de control tiene una variable que cuenta el número de la instancia que se va a enviar en cada momento, y lo utiliza para llamar a la nueva máquina de ese modo. En el primer ataque detectado, la variable aumenta de 1 a 2, para que en el siguiente ataque se utilice el nuevo valor para nombrar la instancia. De este modo, puede tener una idea sin necesidad de mirar el diario del servicio del panel de control de cuantos ataques han sido detectados y evitados. Al final de este apartado se deja un enlace a un video demostrativo de la respuesta del sistema ante dos ataques tanto ejecutado de forma manual como automática y otro enlace al repositorio de github empleado en el proyecto con todos los códigos relevantes utilizados.

El autoescalado coge como argumento la variable descrita, y la usa para nombrar a la nueva instancia. Este código simplemente se encarga de escribir unos logs en un fichero creado en la carpeta /tmp del host que lo ejecuta. La captura a continuación muestra el contenido de este fichero después de que la IA detecte varios ataques.

```
montesponceantalban@vm-db-ai:~/Trabajo-GAR/modulo_ia$ cat /tmp/historial_escalado.txt
[2026-01-10 11:56:08] RECIBIDA ORDEN DE ESCALADO DE EMERGENCIA
[2026-01-10 11:56:08] Contactando con API del Cloud Provider...
[2026-01-10 11:56:10] Desplegando nueva instancia 'web-server-rescue-1'...
[2026-01-10 11:56:12] Instancia iniciada y operativa.
[2026-01-10 11:56:12] Actualizando reglas del Balanceador de Carga...
[2026-01-10 11:56:13] Balanceador reconfigurado. Distribuyendo tráfico.
[2026-01-10 11:56:13] PROTOCOLO DE DEFENSA COMPLETADO CON ÉXITO.
[2026-01-10 11:59:30] RECIBIDA ORDEN DE ESCALADO DE EMERGENCIA
[2026-01-10 11:59:30] Contactando con API del Cloud Provider...
[2026-01-10 11:59:31] Desplegando nueva instancia 'web-server-rescue-1'...
[2026-01-10 11:59:33] Instancia iniciada y operativa.
[2026-01-10 11:59:33] Actualizando reglas del Balanceador de Carga...
[2026-01-10 11:59:34] Balanceador reconfigurado. Distribuyendo tráfico.
[2026-01-10 11:59:34] PROTOCOLO DE DEFENSA COMPLETADO CON ÉXITO.
[2026-01-10 12:03:43] RECIBIDA ORDEN DE ESCALADO DE EMERGENCIA
[2026-01-10 12:03:43] Contactando con API del Cloud Provider...
[2026-01-10 12:03:44] Desplegando nueva instancia 'web-server-rescue-2'...
[2026-01-10 12:03:46] Instancia iniciada y operativa.
[2026-01-10 12:03:46] Actualizando reglas del Balanceador de Carga...
[2026-01-10 12:03:47] Balanceador reconfigurado. Distribuyendo tráfico.
[2026-01-10 12:03:47] PROTOCOLO DE DEFENSA COMPLETADO CON ÉXITO.
[2026-01-10 12:04:10] RECIBIDA ORDEN DE ESCALADO DE EMERGENCIA
[2026-01-10 12:04:10] Contactando con API del Cloud Provider...
[2026-01-10 12:04:11] Desplegando nueva instancia 'web-server-rescue-3'...
[2026-01-10 12:04:13] Instancia iniciada y operativa.
[2026-01-10 12:04:13] Actualizando reglas del Balanceador de Carga...
[2026-01-10 12:04:14] Balanceador reconfigurado. Distribuyendo tráfico.
[2026-01-10 12:04:14] PROTOCOLO DE DEFENSA COMPLETADO CON ÉXITO.
[2026-01-10 12:04:22] RECIBIDA ORDEN DE ESCALADO DE EMERGENCIA
[2026-01-10 12:04:22] Contactando con API del Cloud Provider...
[2026-01-10 12:04:23] Desplegando nueva instancia 'web-server-rescue-4'...
[2026-01-10 12:04:25] Instancia iniciada y operativa.
[2026-01-10 12:04:25] Actualizando reglas del Balanceador de Carga...
[2026-01-10 12:04:26] Balanceador reconfigurado. Distribuyendo tráfico.
[2026-01-10 12:04:26] PROTOCOLO DE DEFENSA COMPLETADO CON ÉXITO.
[2026-01-10 12:04:33] RECIBIDA ORDEN DE ESCALADO DE EMERGENCIA
[2026-01-10 12:04:33] Contactando con API del Cloud Provider...
[2026-01-10 12:04:35] Desplegando nueva instancia 'web-server-rescue-5'...
[2026-01-10 12:04:37] Instancia iniciada y operativa.
[2026-01-10 12:04:37] Actualizando reglas del Balanceador de Carga...
[2026-01-10 12:04:38] Balanceador reconfigurado. Distribuyendo tráfico.
[2026-01-10 12:04:38] PROTOCOLO DE DEFENSA COMPLETADO CON ÉXITO.
```

Enlace video demostrativo: [Funcionamiento final](#)

Enlace al repositorio de github: [Repositorio](#)

5.7 CONCLUSIONES

La realización de este proyecto nos ha permitido validar en la práctica cómo la integración de tecnologías heterogéneas (Inteligencia Artificial, Protocolos de Red y Automatización) es clave para la gestión moderna de infraestructuras críticas. Más allá de la configuración individual de servidores, el verdadero valor del trabajo ha residido en la interconexión de los subsistemas.

Uno de los desafíos técnicos más complejos que enfrentamos fue la disparidad en las frecuencias de muestreo. Inicialmente, el módulo de IA detectaba ataques en microsegundos que pasaban desapercibidos para el ciclo de sondeo (**polling**) del Gestor **SNMP**, que operaba cada segundo o cada 2 segundos.

Aprendimos que en sistemas distribuidos no basta con que los datos sean correctos; deben ser capturables. La implementación de una lógica de histéresis (tiempo de enfriamiento) en el agente y de detección de flancos en el gestor fue determinante para lograr una fiabilidad del 100% en las alertas, evitando falsos negativos y saturación por alertas repetitivas.

Hemos comprobado que SNMP no es un protocolo obsoleto o rígido, sino altamente adaptable. A través del uso de la **MIB** de Extensión (**NET-SNMP-EXTEND-MIB**), logramos convertir una salida no estándar (la predicción de un modelo de Machine Learning en Python) en una métrica gestionable por la red.

Esto nos demuestra que es posible monitorizar cualquier parámetro de negocio (no solo **CPU** o **RAM**) siempre que se pueda encapsular en un script, validando la potencia de las soluciones "Open Source" frente a herramientas propietarias cerradas.

El cierre del bucle de control (Detectar→Alertar→Actuar) ha puesto de manifiesto la importancia de la automatización. Hemos conseguido reducir el tiempo de respuesta ante un incidente de minutos (intervención humana) a segundos (ejecución de scripts). Aunque nuestra solución de autoescalado es una simulación controlada (parada y arranque de servicios), sienta las bases lógicas para implementaciones más complejas con orquestadores como Kubernetes.

Si tuviéramos que llevar este prototipo a un entorno de producción real, identificaríamos las siguientes líneas de mejora:

- Seguridad: Pasar de **SNMP v2c** a **SNMP v3** para cifrar el tráfico de gestión y autenticar a los usuarios, ya que actualmente las alertas viajan en texto plano.
- Escalabilidad Real: Sustituir el reinicio de servicios por un despliegue real de nuevas instancias mediante **Ansible** o **Terraform** en la nube, permitiendo absorber picos de carga reales en lugar de solo reiniciar el servicio afectado.
- Refinamiento de la **IA**: Entrenar el modelo con un dataset de logs reales más extenso para reducir la posibilidad de falsos positivos en entornos de tráfico legítimo variable.

En definitiva, este proyecto ha servido para demostrar que la Alta Disponibilidad no depende solo de tener muchas máquinas, sino de la inteligencia con la que estas se comunican y se gestionan entre sí de forma autónoma.

6 BIBLIOGRAFÍA

Para la búsqueda del dataset para entrenar al modelo:

[Publicly available access.log datasets](#)

[IsolationForest — scikit-learn 1.8.0 documentation](#)

Para la creación de los servicios de monitorización y detección de ataques:

[1] **Red Hat, Inc.** (2024). *System Administrators Guide: Managing Services with Systemd*. Red Hat Enterprise Linux 7 Documentation, Chapter 10. Disponible en:

[Chapter 10. Managing Services with systemd | System Administrator's Guide | Red Hat Enterprise Linux | 7](#)

Red Hat, Inc. (2024). *System Administrators Guide: Managing Services with Systemd*. Red Hat Enterprise Linux 7 Documentation, Chapter 10.6.2 → Para la creación de los servicios y la explicación de los elementos:

[Chapter 10. Managing Services with systemd | System Administrator's Guide | Red Hat Enterprise Linux | 7 | Creating Custom Unit Files](#)

RedesZone. (2021). *Administrar servicios Linux Systemd: Crea un servicio o daemon propio*. → Para la creación de los servicios y la explicación de los elementos:

[Cómo crear o configurar un servicio en servidores Linux usando Systemd](#)

Para el desarrollo del subsistema de monitorización y la configuración de los agentes **SNMP**:

Net-SNMP Project. (s.f.). Net-SNMP Tutorial -- Extending the Agent (exec, sh, extend).

[Tut:Extending snmpd using shell scripts - Net-SNMP Wiki](#)

Estándares del Protocolo (**RFCs**)

Case, J., et al. (1990). *A Simple Network Management Protocol (SNMP)*. RFC 1157. IETF.

Presuhn, R. (2002). *Version 2 of the Protocol Operations for the Simple Network Management Protocol (SNMP)*. RFC 3416. IETF.